

Digital Image Analysis Tutorial 8

```
1  """
2  AI531C T08 - Learn Your First Kernel
3  Run: python T08_studio.py
4
5  Demonstrates:
6  1. 3x3 least-squares kernel learning for deblurring
7  2. 3x3 least-squares kernel learning for denoising
8  3. Patch-count stability
9  4. Optional 5x5 deblurring extension
10
11 This is a self-contained implementation of the T08 concepts.
12 """
13
14 import numpy as np
15 import cv2
16 import matplotlib.pyplot as plt
17
18
19 def psnr(ref, img):
20     ref = ref.astype(float)
21     img = img.astype(float)
22     mse = np.mean((ref - img) ** 2)
23     if mse == 0:
24         return float("inf")
25     return 10 * np.log10(255 ** 2 / mse)
26
27
28 def make_image():
29     """Create a deterministic grayscale test image."""
30     h, w = 256, 256
31     y, x = np.mgrid[0:h, 0:w]
32
33     img = (
34         90
35         + 45 * np.sin(x / 17)
36         + 35 * np.cos(y / 23)
37     ).astype(float)
38
```

```

39     # Add structures/edges so filtering has something to learn.
40     img[45:205, 65:190] += 55
41     img[85:165, 105:150] -= 70
42
43     return np.clip(img, 0, 255).astype(np.uint8)
44
45
46     def patches(image, size=3, max_patches=20000):
47         """Return flattened image patches and their centre-pixel targets."""
48         r = size // 2
49         p = np.pad(image.astype(np.float32), r, mode="reflect")
50
51         X = []
52         y = []
53
54         for row in range(image.shape[0]):
55             for col in range(image.shape[1]):
56                 patch = p[row:row + size, col:col + size]
57                 X.append(patch.reshape(-1))
58                 y.append(image[row, col])
59
60         X = np.asarray(X, dtype=np.float32)
61         y = np.asarray(y, dtype=np.float32)
62
63         if len(X) > max_patches:
64             rng = np.random.default_rng(0)
65             idx = rng.choice(len(X), max_patches, replace=False)
66             X = X[idx]
67             y = y[idx]
68
69         return X, y
70
71
72     def learn_kernel(source, target, size=3, max_patches=20000):
73         """Learn kernel weights using least squares:  $Xw \approx y$ ."""
74         X, y = patches(source, size, max_patches)
75         w, *_ = np.linalg.lstsq(X, y, rcond=None)

```

```

76     return w.reshape(size, size)
77
78
79 def apply(image, kernel):
80     return cv2.filter2D(
81         image.astype(np.float32),
82         -1,
83         kernel.astype(np.float32),
84         borderType=cv2.BORDER_REFLECT_101
85     )
86
87
88 # -----
89 # Create clean reference image
90 # -----
91 clean = make_image()
92
93
94 # =====
95 # ACT 1: DEBLURRING
96 # =====
97
98 blurred = cv2.GaussianBlur(clean, (9, 9), 2.0)
99
100 # Learn a 3x3 kernel: blurred patches -> clean pixels.
101 K_deblur = learn_kernel(blurred, clean, 3, 20000)
102 learned_deblur = np.clip(apply(blurred, K_deblur), 0, 255)
103
104 # Fixed sharpen baseline.
105 K_sharp = np.array([
106     [0, -1, 0],
107     [-1, 5, -1],
108     [0, -1, 0]
109 ], dtype=np.float32)
110
111 fixed_sharpen = np.clip(apply(blurred, K_sharp), 0, 255)
112
113 print("=" * 55)
114 print("ACT 1 - DEBLURRING")

```

```

115 print("=" * 55)
116 print(f"Input blurred:          {psnr(clean, blurred):.2f} dB")
117 print(f"Fixed sharpen:          {psnr(clean, fixed_sharpen):.2f} dB")
118 print(f"Learned 3x3:           {psnr(clean, learned_deblur):.2f} dB")
119
120 print("\nLearned deblur kernel:")
121 print(np.round(K_deblur, 3))
122
123
124 # =====
125 # ACT 2: DENOISING
126 # =====
127
128 rng = np.random.default_rng(1)
129 noise = rng.normal(0, 20, clean.shape)
130 noisy = np.clip(clean.astype(float) + noise, 0, 255).astype(np.uint8)
131
132 # Tuned Gaussian baseline.
133 gaussian = cv2.GaussianBlur(noisy, (5, 5), 1.2)
134
135 # Learn noisy patches -> clean centre pixels.
136 K_denoise = learn_kernel(noisy, clean, 3, 20000)
137 learned_denoise = np.clip(apply(noisy, K_denoise), 0, 255)
138
139 print("\n" + "=" * 55)
140 print("ACT 2 - DENOISING")
141 print("=" * 55)
142 print(f"Noisy input:          {psnr(clean, noisy):.2f} dB")
143 print(f"Tuned Gaussian:       {psnr(clean, gaussian):.2f} dB")
144 print(f"Learned 3x3:           {psnr(clean, learned_denoise):.2f} dB")
145 print(f"Denoise weight sum:   {K_denoise.sum():.4f}")
146
147 print("\nLearned denoise kernel:")
148 print(np.round(K_denoise, 4))
149
150
151 # =====
152 # PATCH COUNT STABILITY
153 # =====

```

```

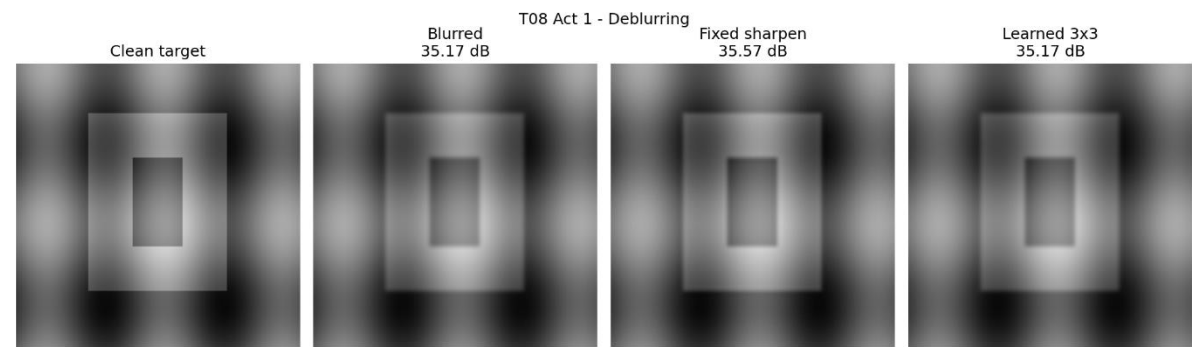
155 print("\n" + "=" * 55)
156 print("ACT 1 - PATCH COUNT STABILITY")
157 print("=" * 55)
158
159 for n in [2000, 5000, 10000, 20000]:
160     K = learn_kernel(blurred, clean, 3, n)
161     result = np.clip(apply(blurred, K), 0, 255)
162     print(f"{n:5d} patches -> {psnr(clean, result):.2f} dB")
163
164
165 # =====
166 # EXTENSION: 5x5 DEBLUR
167 # =====
168
169 K5 = learn_kernel(blurred, clean, 5, 20000)
170 deblur5 = np.clip(apply(blurred, K5), 0, 255)
171
172 print("\n" + "=" * 55)
173 print("EXTENSION - LEARNED 5x5 DEBLUR")
174 print("=" * 55)
175 print(f"Learned 5x5:          {psnr(clean, deblur5):.2f} dB")
176
177
178 # =====
179 # SAVE VISUAL RESULTS
180 # =====
181
182 fig, ax = plt.subplots(1, 4, figsize=(13, 4))
183 items = [
184     (clean, "Clean target"),
185     (blurred, f"Blurred\n{psnr(clean, blurred):.2f} dB"),
186     (fixed_sharpen, f"Fixed sharpen\n{psnr(clean, fixed_sharpen):.2f} dB"),
187     (learned_deblur, f"Learned 3x3\n{psnr(clean, learned_deblur):.2f} dB"),
188 ]
189
190 for a, (im, title) in zip(ax, items):
191     a.imshow(im, cmap="gray", vmin=0, vmax=255)
192     a.set_title(title)
193     a.axis("off")

```

```
194
195 plt.suptitle("T08 Act 1 - Deblurring")
196 plt.tight_layout()
197 plt.savefig("T08_deblur_results.png", dpi=150)
198 plt.close()
199
200
201 fig, ax = plt.subplots(1, 4, figsize=(13, 4))
202 items = [
203     (clean, "Clean target"),
204     (noisy, f"Noisy\n{psnr(clean, noisy):.2f} dB"),
205     (gaussian, f"Tuned Gaussian\n{psnr(clean, gaussian):.2f} dB"),
206     (learned_denoise, f"Learned 3x3\n{psnr(clean, learned_denoise):.2f} dB"),
207 ]
208
209 for a, (im, title) in zip(ax, items):
210     a.imshow(im, cmap="gray", vmin=0, vmax=255)
211     a.set_title(title)
212     a.axis("off")
213
214 plt.suptitle("T08 Act 2 - Denoising")
215 plt.tight_layout()
216 plt.savefig("T08_denoise_results.png", dpi=150)
217 plt.close()
218
219 print("\nDone.")
220 print("Saved: T08_deblur_results.png")
221 print("Saved: T08_denoise_results.png")
```

Output Images:

Deblur result



Denoise result

